

Day 7 — Live coding: Banca Lumera from (almost) nothing

Today we start the assistant this course keeps building: **Banca Lumera**, a retail-banking customer assistant. We build three complete conversational capabilities — balance lookup, branch-appointment booking, card-fees information — and we write **no Python at all**. That is the point of the day: before custom code enters the picture, Rasa already generates a surprising amount of behavior around plain YAML — the asking, the slot-setting, the answering, the repairing. Today is about seeing that machinery run and knowing exactly which file each piece lives in.

The end state of this tutorial is the `lumera-assistant/` folder next to this file: if you get lost, or want to skip ahead, that folder is the working result and it trains and runs as-is.

REQUIREMENTS - `uv` installed (on macOS: `brew install uv`) - a valid `RASA_LICENSE` exported - an `OPENAI_API_KEY` exported (today's template skips the one training-time LLM call, but every conversation needs it)

1. Scaffold from the tutorial template

On Day 6 we scaffolded the **default** template to look around. Today we want the opposite: the least material possible, so that everything in the project ends up being ours. Rasa ships several templates (`default`, `tutorial`, `basic`, `finance`, `telco`); the **tutorial** template is the nearly-empty one.

Start clean:

```
mkdir lumera-assistant && cd lumera-assistant
uv venv --python 3.11 && source .venv/bin/activate
uv pip install rasa-pro
rasa init --template tutorial --init-dir .
```

Accept the training offer. Note something curious against Day 6: this training makes **no LLM call at all** — we will see why in a minute, when we read `config.yml`.

Tour what you got — it fits on one screen:

— actions/actions.py	# one tiny example custom action
— config.yml	# the processing pipeline
— credentials.yml	# channels (how users reach the bot)
— data/flows.yml	# ONE example flow: transfer_money
— data/patterns.yml	# two overridden conversation-repair patterns
— domain.yml	# slots + responses, one flat file
— endpoints.yml	# where things connect (LLM provider, action code)

Compare with Day 6's default template: no `domain/` split directory, no `e2e_tests/`, no mock database — one flow, one flat domain file, one action.

2. Make it ours: remove the toy, repoint the model

2.1 Three removals

The template ships a toy `transfer_money` flow, the Python action it uses, and two overridden repair patterns. None of them is ours; all three go:

```
rm actions/actions.py data/patterns.yml
```

Then empty `data/flows.yml` (we refill it immediately) and delete from `domain.yml` everything except the `version: "3.1"` line — the toy slots and responses served the deleted flow.

Two of those removals deserve a word:

- `actions/actions.py` was the template's one custom action — Python code. We deliberately leave the `actions/` folder in place **empty** (just `__init__.py`): custom actions are Day 8's whole subject, and today's goal is to see how far the assistant goes without them.
- `data/patterns.yml` contained the template's overrides of two built-in repair patterns (`pattern_chitchat`, `pattern_search`). With the file gone, the built-in defaults apply: off-topic messages get a polite "can't help with that". For a bank assistant that stays on task, that default is a reasonable place to start.

2.2 The model indirection: `config.yml` → `endpoints.yml`

Open `config.yml`:

```
recipe: default.v1
language: en
assistant_id: humid-boundary      # yours will differ – a random id per scaffold
pipeline:
- name: CompactLLMCommandGenerator
  llm:
    model_group: rasa_command_generation_model
  flow_retrieval:
    active: false
policies:
- name: FlowPolicy
```

Read the indirection: the pipeline component (`CompactLLMCommandGenerator`, the LLM that turns user messages into commands) does not name a model. It names a **model group** — and the group is resolved in `endpoints.yml`. Look there: the template points that group at a small **Rasa-hosted tutorial model**, free to try, not what we want for real work.

Also note `flow_retrieval: active: false`. Flow retrieval is an embeddings-based index that shortlists relevant flows into the LLM prompt when a project has many of them; the template ships it off, and at our scale (three flows today) off is fine. It is also the answer to the training riddle above: building that index is training's one LLM-API call, so with retrieval off, `rasa train` runs fully offline. Conversations still need the key: every user message goes through the command generator.

Repoint the assistant to OpenAI. In `endpoints.yml`, replace the `model_groups` section (and delete the `nlg` block above it, which belonged to the template's rasa-hosted setup):

```
model_groups:
- id: openai_llm
  models:
  - provider: openai
    model: gpt-5.1-2025-11-13
    reasoning_effort: "none"
    timeout: 15
```

And in `config.yml`, point the component at the new group:

```
llm:
  model_group: openai_llm
```

Notice what is **not** here: the API key. Provider credentials are read from the environment (`OPENAI_API_KEY`) — they never live in a project file. That separation — *which* component in `config.yml`, *where it connects* in `endpoints.yml`, *secrets* in the environment — is the pattern for everything that follows in the course.

3. Flow 1 — `check_balance`: the automatic machinery

The first capability: a customer asks for their balance. One slot, one flow, two responses — and most of the behavior we are about to watch, we will not have written.

3.1 The domain: what the assistant knows and says

In `domain.yml`, add:

```

slots:
  account_type:
    type: categorical
    values:
      - checking
      - savings
    mappings:
      - type: from_llm
  current_balance:
    type: float
    mappings:
      - type: controlled

responses:
  utter_ask_account_type:
    - text: "Which account would you like to check?"
      buttons:
        - title: "Checking"
          payload: "/SetSlots(account_type=checking)"
        - title: "Savings"
          payload: "/SetSlots(account_type=savings)"

  utter_current_balance:
    - text: "Your {account_type} account balance is €{current_balance}."

```

The trust boundary is on display in the two mappings. `account_type` is `from_llm`: the user names it in free text and the LLM fills it. `current_balance` is `controlled`: the LLM can **never** write it — only our own logic can (today a YAML stub, tomorrow real code). An assistant that lets a language model make up account balances is not a bank assistant.

Three more mechanisms, all in the responses block:

- **`utter_ask_account_type` is a naming convention, not a registration.** When a flow needs the `account_type` slot, Rasa runs the response named `utter_ask_<slot name>` automatically. Nothing will reference this response anywhere else — the name *is* the wiring.
- **The buttons carry `/SetSlots` payloads.** A click sends that string instead of free text, and it sets the slot *deterministically* — no model reads it, no interpretation happens. Free text goes through the LLM; buttons bypass it.
- **`utter_current_balance` interpolates slots** with `{curly}` syntax. The response is the answer — there is no "answering code" anywhere.

3.2 The flow: what the assistant can do

Replace the contents of `data/flows.yml`:

```
flows:
  check_balance:
    name: check account balance
    description: Look up the current balance of one of the customer's Banca Lumera
accounts.
    steps:
      - collect: account_type
        description: "the account to check: checking or savings"
      - set_slots:
        - current_balance: 2350.75
      - action: utter_current_balance
```

Read it top to bottom. The flow's `description` is its trigger — it is what the LLM matches user messages against; there is no intent list. The `collect` step asks for the slot (via the auto-run `utter_ask_account_type`) and waits. The `action` step runs the response — responses are legal `action` targets, and for informational flows they are the only "action" you need.

And the middle step: `set_slots` writes `current_balance` directly from YAML. **This line is a lie.** A real bank looks the number up in core banking. We plant the lie deliberately and in plain sight: it is the exact spot where Day 8 replaces YAML with a real system call. (Note it satisfies the `controlled` mapping — `set_slots` and custom actions are the two writers a controlled slot allows.)

3.3 Train, converse, read the commands

```
rasa data validate
rasa train
rasa inspect
```

In the Inspector, walk three interactions and read the right-hand panes as you go:

1. Type **"what's my balance?"** — the tracker shows the command the LLM emitted: **start flow** `check_balance`. Then the machinery: the `collect` step runs `utter_ask_account_type` (you never referenced it — the name did the work), and the two buttons render.
2. **Type** (don't click) **"the savings one, please"** — the LLM emits **set slot** `account_type = savings`: your free text became a structured command. Then `set_slots` fills the balance, and the interpolated response renders: *"Your savings account balance is €2350.75."*
3. Restart the conversation and this time **click the "Checking" button** — same outcome, different path: the `/SetSlots` payload set the slot with no LLM involved. Watch the slot pane: it fills instantly, and no `set slot` command from the model appears.

Count what you wrote versus what ran: two domain entries and a three-step flow, against automatic asking, automatic slot extraction, automatic advancement, and a completion follow-up ("Is there anything else I can help you with?") that comes from a built-in pattern. That ratio is the day's lesson.

4. Flow 2 — `book_appointment` : the multi-slot journey

Second capability: book an in-person appointment at a branch. Now the machinery repeats — city, topic, time — and two new mechanisms appear: a **confirmation gate** and **branching**.

4.1 Split the domain first

With a second feature, the flat `domain.yml` starts to crowd. Rasa accepts a `domain/` **directory** just as happily as a single file, so this is the moment to adopt the layout that scales: one domain file per feature.

```
mkdir domain
```

Move the balance material into `domain/check_balance.yml` (everything from §3.1, plus the `version: "3.1"` header), create `domain/shared.yml` for cross-feature things (it gets content in §6), and delete the flat `domain.yml`. From now on every feature brings its own domain file — the file-hopping rhythm of Rasa development: *domain file* ↔ *flow file*, once per capability.

4.2 The new domain file

```
domain/book_appointment.yml:
```

```

version: "3.1"

slots:
  branch_city:
    type: text
    mappings:
      - type: from_llm
  appointment_topic:
    type: categorical
    values:
      - mortgage
      - investments
      - account services
    mappings:
      - type: from_llm
  preferred_time:
    type: text
    mappings:
      - type: from_llm
  appointment_confirmed:
    type: bool
    mappings:
      - type: from_llm

responses:
  utter_ask_branch_city:
    - text: "In which city would you like the appointment?"

  utter_ask_appointment_topic:
    - text: "What is the appointment about?"
    buttons:
      - title: "Mortgage"
        payload: "/SetSlots(appointment_topic=mortgage)"
      - title: "Investments"
        payload: "/SetSlots(appointment_topic=investments)"
      - title: "Account services"
        payload: "/SetSlots(appointment_topic=account services)"

  utter_ask_preferred_time:
    - text: "When would suit you best? Any day and time, in your own words."

  utter_ask_appointment_confirmed:
    - text: "Let me recap: an appointment at our {branch_city} branch, about {appointment_topic}, {preferred_time}. Shall I book it?"
    buttons:
      - title: "Yes, book it"
        payload: "/SetSlots(appointment_confirmed=true)"
      - title: "No, don't"
        payload: "/SetSlots(appointment_confirmed=false)"

  utter_appointment_booked:
    - text: "Done – your appointment is booked. You'll receive a confirmation shortly."

```

```
utter_appointment_declined:
```

- **text:** "No problem – I've discarded the appointment request."

One deliberate move here: the confirmation question is itself an `utter_ask_<slot>` — asking for the `appointment_confirmed` bool — and its text **interpolates the other three slots**. The recap and the question are one response. Its buttons write `true / false` deterministically; typed answers ("yes please", "actually no") go through the LLM like any other slot.

4.3 The flow, with a branch

Append to `data/flows.yml`:

```
book_appointment:
  name: book a branch appointment
  description: Book an in-person appointment at a Banca Lumera branch.
  steps:
    - collect: branch_city
      description: the city of the branch where the customer wants the appointment
    - collect: appointment_topic
      description: "what the appointment is about: mortgage, investments, or account
services"
    - collect: preferred_time
      description: the customer's preferred day and time, in their own words
    - collect: appointment_confirmed
      description: whether the customer confirms the recap of the appointment
  next:
    - if: slots.appointment_confirmed
      then:
        - action: utter_appointment_booked
          next: END
    - else:
        - action: utter_appointment_declined
          next: END
```

New pieces:

- **next with if / then / else** — the first branch of the course. The predicate (`slots.appointment_confirmed`) is evaluated by the dialogue engine, deterministically — the LLM does not decide which branch runs.
- **Nothing lingers after the flow** — either way the flow ends, and the slots its `collect` steps filled are automatically reset. A declined appointment leaves no stale city or topic behind: a second attempt re-asks everything. (If a value *should* survive the flow, that is an explicit design decision — a topic for the day on state at scale.)
- Every collect gets a `description`: it sharpens what the LLM extracts (compare `preferred_time`'s "in their own words" — we accept free-form text on purpose today).

Retrain and converse (`rasa train`, then `rasa inspect`). A full run:

user: I'd like to book an appointment at a branch **bot:** In which city would you like the appointment? **user:** Napoli **bot:** What is the appointment about? (+ *three buttons*) **user:** it's about a mortgage **bot:** When would suit you best? Any day and time, in your own words. **user:** next Tuesday morning **bot:** Let me recap: an appointment at our Napoli branch, about mortgage, next Tuesday morning. Shall I book it? (+ *Yes/No buttons*) **user:** yes please, book it **bot:** Done — your appointment is booked. You'll receive a confirmation shortly.

Run it a second time and decline at the recap: you get the decline response, and — try booking again — the city question comes back fresh: the slots were reset when the flow ended.

Honesty check, same as the balance stub: **nothing was booked anywhere**. The confirmation is conversational only. That, too, is a Day 8 hole, planted on purpose.

4.4 A behavior you didn't write: correction

While a booking is in progress, change your mind:

user: I want a branch appointment **bot:** In which city would you like the appointment? **user:** Napoli **bot:** What is the appointment about? (...) **user:** actually, make it the Milano branch instead **bot:** Ok, I am updating branch_city to Milano. **bot:** What is the appointment about? (...)

Nobody wrote a correction handler. The LLM emitted a `set slot` for an *already-filled* slot, and a built-in repair pattern acknowledged the change and resumed where the flow stood — the recap at the end will say Milano. This is the pattern machinery working underneath every flow you define; you get it for free.

5. Flow 3 — `card_fees_info` : many right answers for one question

Third capability, honestly backend-free this time: card fees are reference information, so **the responses really are the whole answer**. The new mechanism: one response name, several variants, chosen by conversation state.

5.1 Conditional response variations

`domain/card_fees_info.yml` :

```

version: "3.1"

slots:
  card_type:
    type: categorical
    values:
      - debit
      - credit
    mappings:
      - type: from_llm

responses:
  utter_ask_card_type:
    - text: "Which card are you interested in?"
      buttons:
        - title: "Debit card"
          payload: "/SetSlots(card_type=debit)"
        - title: "Credit card"
          payload: "/SetSlots(card_type=credit)"

  utter_card_fees:
    - condition:
        - type: slot
          name: card_type
          value: debit
      text: "The Lumera debit card has no yearly fee – it comes free with every current account."
    - condition:
        - type: slot
          name: card_type
          value: credit
      text: "The Lumera credit card costs €45 per year, waived if you spend more than €6,000 in the year."
    - text: "Every Lumera card has its own fee schedule, listed in the full fee table."

  utter_card_fee_table_link:
    - text: "Full details for your card: https://banca-lumera.example/cards/{card_type}/fees"

```

`utter_card_fees` is **one response with three variations**. Each `condition` block matches on a slot value; the engine picks the first variant whose condition holds, and the last, condition-free variant is the fallback — always provide one. No LLM chooses the wording: these are exact, brand-approved sentences, selected deterministically.

And `utter_card_fee_table_link` carries a **deeplink with a slot inside the URL**: `{card_type}` interpolates into the link itself. A flow can end in exactly the kind of parameterized deeplink a search product returns — with the wording and the URL fully under your control.

5.2 The flow

Append to `data/flows.yml`:

```

card_fees_info:
  name: card fees information
  description: Explain the yearly fees of Banca Lumera payment cards.
  steps:
    - collect: card_type
      description: "the card the customer asks about: debit or credit"
    - action: utter_card_fees
    - action: utter_card_fee_table_link

```

Retrain and try: **"how much does the credit card cost per year?"** — notice the collect step *doesn't* ask: the LLM already extracted `card_type=credit` from your opening message, so the flow sails straight to the answer:

bot: The Lumera credit card costs €45 per year, waived if you spend more than €6,000 in the year. **bot:** Full details for your card: <https://banca-lumera.example/cards/credit/fees>

A `collect` step only asks when the slot is empty. Ask generically ("tell me about card fees") and it does ask, buttons and all.

5.3 The Italian variation

Banca Lumera serves Italian customers; the assistant's replies should exist in Italian. Two pieces. First, declare the language in `config.yml` (top level, next to `language`):

```

language: en
additional_languages:
  - it

```

Second, each response carries its translations inline, under a `translation` key — the top-level `text` is the default language. Update the three `utter_card_fees` variants and the link response in `domain/card_fees_info.yml`:

```

utter_card_fees:
- condition:
  - type: slot
    name: card_type
    value: debit
  text: "The Lumera debit card has no yearly fee – it comes free with every current account."
  translation:
    it: "La carta di debito Lumera non ha alcun canone annuo: è inclusa in ogni conto corrente."
- condition:
  - type: slot
    name: card_type
    value: credit
  text: "The Lumera credit card costs €45 per year, waived if you spend more than €6,000 in the year."
  translation:
    it: "La carta di credito Lumera costa 45 € l'anno, azzerati se spendi più di 6.000 € nell'anno."
- text: "Every Lumera card has its own fee schedule, listed in the full fee table."
  translation:
    it: "Ogni carta Lumera ha il proprio piano commissioni, riportato nella tabella completa."

utter_card_fee_table_link:
- text: "Full details for your card: https://banca-lumera.example/cards/{card_type}/fees"
  translation:
    it: "Tutti i dettagli per la tua carta: https://banca-lumera.example/cards/{card_type}/fees"

```

Which language a conversation runs in is held by a **built-in language slot** (a reserved name — you cannot declare a slot called `language` yourself). Setting it is *your* application's job — from a stored preference, a channel parameter, a browser locale — and that is code, so it properly belongs to Day 8. For a quick local test, though, the deterministic door we already know works here too — send this message in the Inspector:

```
/SetSlots(language=it)
```

...then ask about the debit card:

bot: La carta di debito Lumera non ha alcun canone annuo: è inclusa in ogni conto corrente. **bot:** Tutti i dettagli per la tua carta: https://banca-lumera.example/cards/debit/fees **bot:** Is there anything else I can help you with?

Read the third line carefully: it is still English. Only responses that *have* an Italian translation are served in Italian — the built-in pattern responses have none. Translation is a completeness exercise, and Rasa makes completeness checkable:

```
rasa data validate translations
```

warns, response by response, about every missing `it` translation — our seven still-untranslated responses *and* every built-in one. (Plain `rasa data validate` shows the same as two aggregate counts.) We leave the rest untranslated today; the mechanism is what matters.

6. Session configuration

One last knob while we are in the domain: how long a conversation "session" lives. In `domain/shared.yml`:

```
version: "3.1"

session_config:
  session_expiration_time: 60 # minutes of inactivity before a new session starts
  carry_over_slots_to_new_session: true
```

These are the defaults, written out so they are visible: after 60 minutes of silence a returning user starts a fresh session, and slot values carry over into it. Whether carried-over slots are *right* for every value (should an old `appointment_confirmed` survive?) is a design question we will meet again when state management gets its own day.

7. Where we landed

Final layout — compare it with the one-screen scaffold we started from:

```
|— actions/                # EMPTY on purpose – Day 8's subject
|— config.yml              # + additional_languages, openai_llm group ref
|— credentials.yml
|— data/flows.yml          # three flows, all YAML
|— domain/
|   |— shared.yml          # session config
|   |— check_balance.yml
|   |— book_appointment.yml
|   |— card_fees_info.yml
|— endpoints.yml           # openai_llm model group; key stays in the env
```

Three capabilities, zero Python. What ran that we never wrote: automatic asking (`utter_ask_<slot>`), automatic extraction (`from_llm` + `SetSlot` commands), deterministic button input (`/SetSlots`), automatic flow advancement, correction handling, completion follow-ups, off-topic fallback, per-state response selection, translation serving.

And two honest lies, planted in plain sight, both waiting for Day 8:

1. `set_slots: current_balance: 2350.75` — the balance is invented in YAML; no bank system was consulted.

2. "Done — your appointment is booked" — nothing was booked anywhere.

Day 8 replaces both with real calls into a (mock) core-banking API — and that is where Python finally enters the project.

8. Extra — turning chitchat on (demo only, not kept)

Extra. This section is a live demo, not part of the project. We revert both edits at the end; the Day 8 snapshot starts without them.

Two things we deleted earlier get their explanation now: the `data/patterns.yml` overrides from §2.1, and the `nlg:` block from §2.2. Try the assistant first: say something off-topic — "do you like pizza?" — and the built-in `pattern_chitchat` deflects with a polite "can't help with that". That deflection is a flow Rasa ships; like any pattern, we can override it by defining a flow with the same name. Recreate `data/patterns.yml`:

```
flows:
  pattern_chitchat:
    description: handle interactions with the user that are not task-oriented
    name: pattern chitchat
    steps:
      - action: utter_free_chitchat_response
```

Flows are trained artifacts, so this needs a `rasa train`. Ask about pizza again — and the answer is a *fixed string*: "Sorry, I'm not able to answer that right now." Odd, for a response named "free chitchat". The reason: the built-in `utter_free_chitchat_response` is marked `rephrase: True` in its metadata, but rephrasing is done by a component we don't have — the **contextual response rephraser**, an LLM that rewrites templated responses in context. It lives in the response layer, which is neither the `pipeline:` nor the `policies:` — it is configured in `endpoints.yml`, in exactly the `nlg:` slot we deleted in §2.2:

```
nlg:
  type: rephrase
```

No retraining — endpoints are runtime wiring, so restart the server and ask about pizza once more. Now the assistant actually chats back, in context. Read what just traversed the whole architecture: the **pipeline** (command generator) classified the message as `Chitchat`, the **policies** (FlowPolicy) ran `pattern_chitchat`, and the **NLG layer** generated the words. Three stages, three config homes.

Why we don't keep it: free-form generation means the LLM may happily answer far outside banking — the docs warn exactly this. For an assistant that should stay on task, the deflecting default is the right call, so delete `data/patterns.yml` and the `nlg:` block again, and retrain. We will meet `pattern_chitchat` once more, later in the course, when we break the assistant on purpose and freeze its repairs into tests.